DevOps (Lab)

Key Concepts of GitHub Actions

- **Workflow**: A YAML file that defines a set of jobs to be executed when specific events occur (e.g., `push`, `pull_request`).
- **Job**: A collection of steps that run on a specified virtual environment (e.g., `ubuntu-latest`).
- **Step**: An individual task within a job, such as running a script or executing an action.
- **Actions**: Predefined or custom reusable commands.
- **Secrets**: Encrypted variables used to secure sensitive information (e.g., API keys).

Running Jobs in GitHub Actions

In GitHub Actions, jobs are the primary unit of work in a workflow. Jobs can be executed either **in parallel** or **sequentially** depending on the configuration in the YAML file.

Parallel Execution of Jobs

By default, jobs in a GitHub Actions workflow run **in parallel**, meaning they will start at the same time and run concurrently, provided they do not have dependencies on each other.

This is useful for tasks that are independent of each other, allowing you to speed up your CI/CD pipeline by running jobs simultaneously.

Example (Parallel Execution):

```yaml
name: Parallel Jobs Example

on: [push]

jobs:
  job1:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Job 1"

  job2:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Job 2"

  job3:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Job 3"
```

In the example above, `job1`, `job2`, and `job3` will run in parallel because they do not have any dependencies defined between them.

Sequential Execution of Jobs

To run jobs sequentially, meaning one job starts only after the completion of another, you can use the `needs` keyword. This allows you to define a dependency between jobs, where a job will only execute after another job successfully finishes.

Example (Sequential Execution):

```yaml
name: Sequential Jobs Example

on: [push]

jobs:
  job1:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Job 1"

  job2:
    runs-on: ubuntu-latest
    needs: job1  # job2 will start after job1 finishes
    steps:
      - run: echo "Job 2"

  job3:
    runs-on: ubuntu-latest
    needs: job2  # job3 will start after job2 finishes
    steps:
      - run: echo "Job 3"
```

In this case:

- `job1` runs first.
- `job2` starts only after `job1` completes successfully.
- `job3` starts only after `job2` completes successfully.

Combining Parallel and Sequential Jobs

You can also combine parallel and sequential jobs in a single workflow. Some jobs may need to run sequentially (due to dependencies), while others can run in parallel.

Example (Parallel and Sequential Combination):

```yaml
name: Mixed Jobs Example

on: [push]

jobs:
  job1:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Job 1"

  job2:
    runs-on: ubuntu-latest
    needs: job1  # Runs after job1
    steps:
      - run: echo "Job 2"

  job3:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Job 3"

  job4:
    runs-on: ubuntu-latest
    needs: [job2, job3]  # Runs after both job2 and job3
    steps:
      - run: echo "Job 4"
```

In this case:

- `job1` runs first.
- `job3` runs in parallel with `job1`.
- `job2` starts after `job1` completes.
- `job4` runs only after both `job2` and `job3` finish.

Continuous Delivery Workflow

The following configuration file defines a CI/CD pipeline for a Java project. It includes steps for linting, security scanning, building, and deploying a Docker image. Here's how to assemble it step by step.

Define Workflow Metadata

Create a file under the repository path *repo_root/.github/workflows/workflow.yml*. Each file in this directory defines a separate workflow, and workflows are triggered and executed independently in parallel when their respective events occur.

Start with the workflow's name and triggers:

```yaml
name: CI-CD Workflow

on:
  push:
    branches: [ "main" ] # Trigger on pushes to the main branch
  pull_request:
    branches: [ "main" ] # Trigger on pull requests targeting the main branch
```

- **`name`** : Descriptive title for the workflow.
- **`on`** : Specifies the events that trigger the workflow (`push`, `pull_request`).

Add Jobs Section

Jobs define the tasks to execute. Each job runs independently unless dependencies are specified. In this example, we define:

1. **Lint Scan**: Checks code quality using Super-Linter.
2. **Build Artifact**: Builds a Java artifact using Maven.
3. **Build and Push Docker Image**: Builds and pushes a Docker image.

Define the `lint-scan` Job

Super Linter is a powerful GitHub Action that helps automate the process of linting code in various programming languages. It is designed to detect syntax and style issues in your code, ensuring that it adheres to best practices and code standards. The Super Linter action supports a wide range of languages, including Python, JavaScript, Java, Ruby, Go, and more, making it an excellent choice for multi-language projects.

- **Multi-Language Support**: Super Linter supports dozens of programming languages, helping developers ensure consistent code quality across various files.
- **Customizable**: You can configure Super Linter to use specific linters for different languages, providing flexibility to match your project's needs.
- **GitHub Integration**: It seamlessly integrates with GitHub Actions, enabling automatic linting as part of your CI/CD pipeline.
- **Easy Setup**: With just a few lines in your GitHub Actions workflow file, you can set up Super Linter and start linting your code.

```yaml
  # Lint Scan Job
  lint-scan:
    runs-on: ubuntu-latest # Use the latest Ubuntu runner

    permissions:
      contents: read # Required to access repository contents
      packages: read # Required to read packages
      statuses: write # Allow reporting GitHub status checks

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
        with:
          fetch-depth: 0 # Fetch full history for accurate file change tracking

      - name: Run Super-Linter
        uses: super-linter/super-linter@v7.3.0
        env:
          VALIDATE_JAVA: true # Enables checks for Java and disables all others
          GITHUB_TOKEN: ${{ secrets.TOKEN }} # Authentication token
```

- **`runs-on`** : Specifies the virtual environment (e.g., Ubuntu).
- **Steps**:
  - **Checkout code**: Fetches the repository contents.
  - **Run Super-Linter**: Validates Java files using a linter.

Add the `build-artifact` Job

```yaml
  # Build Artifact Job
  build-artifact:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        java-version: [21, 22] # Test against Java 21, 22

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Java Environment
        uses: actions/setup-java@v4
        with:
          java-version: ${{ matrix.java-version }} # Use the version from the matrix
          distribution: 'temurin'
          cache: maven # Enable Maven dependency caching

      - name: Build with Maven
        run: mvn --batch-mode --update-snapshots verify

      - name: Copy Build Artifact
        run: mkdir staging && cp target/*.jar staging

      - name: Upload Build Artifact
        uses: actions/upload-artifact@v4
        with:
          name: jar-artifact-java-${{ matrix.java-version }} # Make the artifact name include the Java version
          path: staging
```

This job builds the project and saves the JAR artifact.

Add the `build-push-container-image` Job

```yaml
  # Build and Push Docker Image Job
  build-push-container-image:
    runs-on: ubuntu-latest
    needs: build-artifact # Depend on the artifact build job

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Download Build Artifact
        uses: actions/download-artifact@v4
        with:
          name: jar-artifact-java-21
          path: target/

      - name: Build Docker Image
        run: docker build -t nbicocchi/product-service-ci-cd:latest .

      - name: Log in to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_USERNAME }} # Docker Hub username
          password: ${{ secrets.DOCKER_PASSWORD }} # Docker Hub password

      - name: Push Docker Image to Docker Hub
        run: docker push nbicocchi/product-service-ci-cd:latest
```

This job builds a Docker image, scans it for vulnerabilities using Trivy, and pushes it to Docker Hub.

Security Workflow

GitGuardian: A Secret Detection Tool

GitGuardian is a security tool designed to detect sensitive information (secrets) such as API keys, passwords, and tokens in source code repositories. It helps organizations secure their codebases by preventing accidental leaks of confidential data.

- **Secret Detection**: Identifies hardcoded secrets like API keys, access tokens, and private credentials that may have been inadvertently committed to version control.
- **Real-Time Scanning**: Continuously scans repositories for secrets, either on push or in pull requests, to ensure that sensitive data does not get exposed.
- **Comprehensive Coverage**: Supports a variety of file types and languages, detecting secrets across code, configuration files, and even documentation.
- **Integrations**: Easily integrates with GitHub, GitLab, and other version control systems, as well as CI/CD pipelines, to provide automated secret detection.

```yaml
  # GitGuardian Secrets Scan Job
  gitguardian-scan:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: GitGuardian Scan
        uses: GitGuardian/ggshield-action@v1
        env:
          GITHUB_PUSH_BEFORE_SHA: ${{ github.event.before }}
          GITHUB_PUSH_BASE_SHA: ${{ github.event.base }}
          GITHUB_DEFAULT_BRANCH: ${{ github.event.repository.default_branch }}
          GITGUARDIAN_API_KEY: ${{ secrets.GITGUARDIAN_API_KEY }}
```

- Create a service account from the API section of your GitGuardian workspace (or a personal access token if you are on the Free plan).
- Add this API key to the **GITGUARDIAN_API_KEY** secret in your project settings.

Semgrep: A Static Code Analysis Tool

Semgrep is a powerful, fast, and flexible static code analysis tool designed to find vulnerabilities, enforce coding standards, and detect patterns in codebases. It is used for security scanning, bug hunting, and code quality checks.

- **Pattern Matching**: Allows users to define custom patterns for detecting specific code issues, such as security vulnerabilities, anti-patterns, or code smells.
- **Multi-Language Support**: Works with a wide range of programming languages, including Python, JavaScript, Go, Java, and many more.
- **High Customizability**: Users can write their own rules or use community-contributed rules tailored for various security concerns or coding standards.
- **Integration with CI/CD**: Easily integrates into CI/CD pipelines for continuous security and quality checks during code development and deployment.

```yaml
semgrep-scan:
  runs-on: ubuntu-latest
  container:
    image: semgrep/semgrep

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Run Semgrep Scan
      run: semgrep ci
      env:
        SEMGREP_APP_TOKEN: ${{ secrets.SEMGREP_APP_TOKEN }}
```

- Create a new API token in the Semgrep dashboard.
- Add this API key to the **SEMGREP_APP_TOKEN** secret in your project settings.

Trivy: A Vulnerability Scanning Tool

Trivy is a versatile and easy-to-use security tool designed to detect vulnerabilities in software components. It is widely used in DevOps workflows for ensuring secure deployments.

- **Vulnerability Detection**: Scans operating system packages and application dependencies for known security vulnerabilities.
- **Container Image Scanning**: Identifies vulnerabilities in Docker images, including both OS-level issues and library dependencies.
- **Infrastructure as Code (IaC) Scanning**: Examines IaC configurations (e.g., Terraform, Kubernetes manifests) for misconfigurations.
- **Versatile Output Formats**: Supports detailed reports in table, JSON, or other formats for integration with CI/CD pipelines.

```yaml
# Build and Push Docker Image Job
build-push-container-image:
  runs-on: ubuntu-latest
  needs: build-artifact # Depend on the artifact build job

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Download Build Artifact
      uses: actions/download-artifact@v4
      with:
        name: jar-artifact-java-21
        path: target/

    - name: Build Docker Image
      run: docker build -t nbicocchi/product-service-ci-cd:latest .

    - name: Run Trivy Vulnerability Scan
      uses: aquasecurity/trivy-action@0.28.0
      with:
        image-ref: 'docker.io/nbicocchi/product-service-ci-cd:latest'
        format: 'table'
        exit-code: '1' # Fail the job if critical issues are found
        ignore-unfixed: true
        vuln-type: 'os,library'
        severity: 'CRITICAL,HIGH'

    - name: Log in to Docker Hub
      uses: docker/login-action@v2
      with:
        username: ${{ secrets.DOCKER_USERNAME }} # Docker Hub username
        password: ${{ secrets.DOCKER_PASSWORD }} # Docker Hub password

    - name: Push Docker Image to Docker Hub
      run: docker push nbicocchi/product-service-ci-cd:latest
```

Infrastructure Provisioning Workflow

The *Infrastructure Provisioning* workflow allows developers or DevOps engineers to automatically create and configure infrastructure resources (e.g., virtual machines, networks, security groups) on AWS using Terraform scripts.

```yaml
name: Infrastructure provisioning

on:
  workflow_dispatch:

jobs:
  infrastructure:
    runs-on: ubuntu-latest

    permissions:
      contents: read

    environment:
      name: Infrastructure provisioning

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: eu-north-1

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.13.3

      - name: Terraform init
        id: init
        run: terraform init -input=false
        working-directory: terraform

      - name: Terraform plan
        id: plan
        run: terraform plan -input=false
        working-directory: terraform

      - name: Terraform apply
        id: apply
        run: terraform apply -input=false -auto-approve
        working-directory: terraform

      - name: Get OpenSSH private key
        id: get_key
        run: |
          PRIVATE_KEY=$(terraform output -raw private_key_openssh)
          echo "$PRIVATE_KEY"
        working-directory: terraform
```

- **AWS Credential Configuration**: The workflow authenticates to AWS using the `aws-actions/configure-aws-credentials` action. Credentials are securely stored in GitHub Secrets (`AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`) to prevent leaks.
- **Terraform Setup**: The action `hashicorp/setup-terraform` installs a specific Terraform version (`1.13.3`) on the runner, ensuring consistent infrastructure provisioning across executions.
- **Terraform Commands**:
  - `terraform init` : Initializes the Terraform working directory, downloading required providers and modules.
  - `terraform plan` : Creates and displays an execution plan, showing the changes Terraform will apply.
  - `terraform apply` : Executes the plan and provisions or updates infrastructure automatically (`-auto-approve` skips manual confirmation).
- **Retrieving SSH Key**: After provisioning, the workflow extracts the OpenSSH private key output from Terraform (via `terraform output -raw private_key_openssh`). This key can be used to connect securely to the provisioned virtual machine.

Docker Setup Workflow

This workflow automates the **remote configuration of Docker** on an **AWS EC2 instance** using **Ansible**. It securely connects to a provisioned virtual machine, installs and configures Docker, and prepares the environment for container-based deployments.

```yaml
name: Docker setup

on:
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure SSH key for ec2 access
        run: |
          mkdir -p ~/.ssh
          chmod 700 ~/.ssh
          printf "%s\n" "${{ secrets.EC2_SSH_KEY }}" > ~/.ssh/deploy_key
          chmod 600 ~/.ssh/deploy_key

      - name: Add ec2 IP to known host list
        run: |
          ssh-keyscan -H ${{ secrets.EC2_HOST }} >> ~/.ssh/known_hosts || true

      - name: Set up Ansible
        run: sudo apt-get update && sudo apt-get install -y ansible

      - name: Run Docker setup playbook
        run: |
          ansible-playbook \
          -i ${{ secrets.EC2_HOST }}, ansible/docker-setup.yml \
          --user ${{ secrets.EC2_USER }} \
          --private-key ~/.ssh/deploy_key
```

- **Trigger (`workflow_dispatch`)**: The workflow runs only when triggered manually, ensuring that Docker installation occurs under explicit operator control.
- **SSH Configuration**: The SSH key required for connecting to the EC2 instance is securely injected into the GitHub runner.
  - The private key (`EC2_SSH_KEY`) is retrieved from GitHub Secrets.
  - The EC2 host's public key is added to the `known_hosts` list to avoid SSH authenticity prompts.
- **Ansible Setup**: Installs Ansible on the GitHub Actions runner, enabling remote automation via playbooks.
- **Run Ansible Playbook**: Executes the playbook `ansible/docker-setup.yml`, which connects to the remote EC2 instance using the provided credentials (`EC2_USER` and SSH key). The playbook is responsible for:
  - Installing Docker and related dependencies.
  - Configuring user permissions for Docker.
  - Ensuring Docker service is enabled and started.

Docker Run Workflow

This workflow automates the **deployment and execution of Docker containers** on a remote **AWS EC2 instance** using **Ansible**. It builds upon the previous *Docker Setup* workflow, assuming Docker is already installed and configured on the target machine.

```yaml
name: Docker run

on:
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Configure SSH key for ec2 access
        run: |
          mkdir -p ~/.ssh
          chmod 700 ~/.ssh
          printf "%s\n" "${{ secrets.EC2_SSH_KEY }}" > ~/.ssh/deploy_key
          chmod 600 ~/.ssh/deploy_key

      - name: Add ec2 IP to known host list
        run: |
          ssh-keyscan -H ${{ secrets.EC2_HOST }} >> ~/.ssh/known_hosts || true

      - name: Set up Ansible
        run: sudo apt-get update && sudo apt-get install -y ansible

      - name: Run Docker deploy playbook
        run: |
          ansible-playbook \
          -i ${{ secrets.EC2_HOST }}, ansible/docker-run.yml \
          --user ${{ secrets.EC2_USER }} \
          --private-key ~/.ssh/deploy_key
```

- **Manual Trigger (`workflow_dispatch`)**: The workflow is executed on demand, providing controlled deployment of containers when a new version of an image or configuration needs to be deployed.
- **Secure SSH Configuration**:
  - Retrieves the private SSH key (`EC2_SSH_KEY`) from GitHub Secrets and configures it on the runner.
  - Adds the EC2 host's SSH fingerprint to the `known_hosts` list to avoid interactive authentication prompts. This ensures a **non-interactive, secure connection** to the target host.
- **Ansible Setup**: Installs Ansible on the GitHub Actions runner, enabling infrastructure automation without requiring preinstalled dependencies.
- **Ansible Playbook Execution**: Runs the playbook `ansible/docker-run.yml`, which typically includes tasks such as:
  - Pulling the latest Docker image from a container registry (e.g., Docker Hub, ECR).
  - Stopping and removing any previous container instances.
  - Running new containers with updated images or environment configurations.
  - Optionally managing Docker Compose services or performing health checks.